

Packing Squares into Rectangles in Pychoco

Luca Davì 75214

NOVA School of Science and Technology

1 Problem Description

Rectangle Packing problems involve packing all squares with sizes 1×1 to $n \times n$ into an enclosing rectangle of minimum area. Rectangle packing has applications in a variety of real-world settings, e.g. packing of blocks into a circuit layout and scheduling.

The objective of this report is to illustrate the modeling and implementation of the Rectangle Packing problem as a Constraint Satisfaction Problem (CSP) using Pychoco, a Python wrapper for the Choco constraint solver. We discuss the choice of constraints, symmetry breaking, and search strategies, and evaluate their impact on solver performance.

2 Constraint Programming Model

We used the established constraint model for the rectangle packing problem. Each square to be placed is defined by a pair of decision variables X and Y , and two integer constants W and H representing its dimensions. In the case of packing squares, W and H are identical.

2.1 Variables Definitions and Domains

For each square $i \in \{1, \dots, n\}$ of size $i \times i$, we define two integer decision variables:

- $x_i \in \{0, \dots, \text{maxW} - i\}$ - the x-coordinate of the bottom-left corner,
- $y_i \in \{0, \dots, \text{maxH} - i\}$ - the y-coordinate of the bottom-left corner.

The upper bounds $\text{maxW} - i$ and $\text{maxH} - i$ enforce that each square lies entirely within the rectangle. The widths and heights are fixed parameters $w_i = h_i = i$, represented as singleton *intvars*.

2.2 Constraints

Non-overlap between all pairs of squares is enforced by the global constraint $\text{diff_n}(X, Y, W, H)$, which ensures that no two rectangles share any area. Compared to a decomposition in disjunctive constraints, diff_n achieves stronger propagation by reasoning globally over all rectangles.

2.3 Symmetry Breaking Constraints

A rectangle of dimensions $\text{maxW} \times \text{maxH}$ presents four symmetric solutions: the original placement, a horizontal reflection, a vertical reflection, and their composition. To eliminate redundant solutions, the largest square is restricted to the bottom-left quadrant of the rectangle by imposing $x_n = 0 \wedge y_n = 0$ via two *arithm* constraints.

3 Implementation

3.1 PACK Function

The CP model is implemented in the `pack(n, maxW, maxH)` function. This function declares variables and domains, posts the constraints described in Section 2, sets the search strategy, and calls the solver. It returns a $2 \times n$ matrix of origin coordinates:

$$\left[[x_i]_{i=1}^n, [y_i]_{i=1}^n \right]$$

If no solution is found, the matrix is filled with zeros.

3.2 BEST_FITTING_RECT Function

The search for the minimum-area rectangle is performed by the `best_fitting_rect(n)` function, which enumerates candidate rectangle dimensions (maxW, maxH) in order of increasing area and calls `pack` on each until a solution is found.

Candidates are generated considering two lower bounds. For a fixed width maxW , the height $\text{maxH} \leq \text{maxW}$ must satisfy:

$$\text{maxH} \geq n \tag{1}$$

$$\text{maxH} \geq \left\lceil \frac{\sum_{i=1}^n i^2}{\text{maxW}} \right\rceil = \left\lceil \frac{n(n+1)(2n+1)}{6 \cdot \text{maxW}} \right\rceil \tag{2}$$

where $\text{maxH} \geq n$ ensures the largest square fits vertically, and the area bound ensures the rectangle is large enough to contain all squares. Taking the maximum of the two bounds gives the tightest lower bound:

$$\text{maxH} \geq \max \left(n, \left\lceil \frac{n(n+1)(2n+1)}{6 \cdot \text{maxW}} \right\rceil \right)$$

with maxW ranging over $\{n, \dots, \frac{n(n+1)(2n+1)}{6}\}$ and $\text{maxH} \leq \text{maxW}$ ensuring that only one orientation of each rectangle is considered.

Candidates are sorted by area before the search begins. Since `pack` is called in this order, the first solution returned is guaranteed to have the minimum enclosing area.

4 Search Strategy

The *dom/wdeg* heuristic [1] was selected as the search strategy, based on its performance observed during experimentation, as discussed in Section 5.

Dom/wdeg is a look-back heuristic that maintains a weight for each constraint, incremented each time that constraint is involved in a failure. The variable selected for branching at each node is the one minimising the ratio:

$$\frac{|\text{dom}(x_i)|}{\text{wdeg}(x_i)}$$

where $|\text{dom}(x_i)|$ is the current domain size of variable x_i and $\text{wdeg}(x_i)$ is the sum of the weights of all constraints in which x_i participates. A lower ratio indicates a variable that is tightly constrained and heavily involved in past failures. The value ordering used is lowest bound first.

Three alternative heuristics were also evaluated: the default search offered by Psychoco, the activity-based heuristic, and the input-order heuristic with largest-square-first ordering, as suggested by [2].

5 Experimental Results

All experiments were conducted on an Apple Macbook Air (M4, 2025), running Python 3.13.7 and Psychoco 0.2.6.

5.1 Search Strategy Comparison

Table 1 reports the solver statistics for $n = 15$ under the four strategies evaluated.

Table 1. Search strategy comparison for $n = 15$, rectangle 55×23

Strategy	Failures	Backtracks	Restarts	Time (s)
Default	61	117	0	0.0021
Input order	82	160	0	0.0018
<i>dom/wdeg</i>	24	34	0	0.0009
Activity-based	2006	6395	1217	0.0414

Dom/wdeg achieves the fewest failures and the lowest solving time, confirming its value as a search strategy for this problem. The activity-based heuristic incurs a significantly higher cost due to the overhead of its restart-based learning phase. The input-order heuristic provides no benefit over the default search, suggesting that a static largest-first ordering, even if tailored for the problem, performs worse than dynamic strategies.

The overall computational cost of `best_fitting_rect` is determined not only by the complexity of each `pack` call, but also by the number of candidate

rectangles that must be evaluated. For increasing values of n , both the number of candidates and the cost of each call grow, making the search progressively more expensive.

5.2 Minimum Rectangle Results

Table 2 reports the minimum enclosing rectangle found for each value of n , along with the corresponding fill percentage defined as the ratio of total square area to rectangle area, and the number of candidates generated.

It is worth noting that the number of candidates represents an upper-bound on the number of instances evaluated.

Table 2. Minimum enclosing rectangle for $n = 5$ to 17

n	maxW	maxH	Area	Fill (%)	Candidates	Solve time (s)	Total time (s)
5	12	5	60	91.7	1,315	0.0007	0.0145
6	11	9	99	91.9	3,719	0.0001	0.0013
7	14	11	154	90.9	9,009	0.0001	0.0030
8	15	14	210	97.1	19,443	0.0002	0.0036
9	20	15	300	95.0	38,412	0.0007	0.0087
10	27	15	405	95.1	70,739	0.0002	0.0161
11	27	19	513	98.6	123,063	0.0104	0.0514
12	29	23	667	97.5	204,214	0.0040	0.1117
13	38	22	836	98.0	325,673	0.0011	0.1822
14	45	23	1035	98.1	502,039	0.0014	1.6892
15	55	23	1265	98.0	751,558	0.0008	5.3594
16	54	28	1512	98.9	1,096,671	0.2054	29.9780
17	46	39	1794	99.5	1,564,632	2.1867	47.4031

Fill percentage is consistently high and approaches near-perfect packing for larger instances, such as $n = 17$ at 99.5%. The total solving time grows significantly with n , driven by the increase in the number of candidates to evaluate, rather than the difficulty of individual `pack` call, most of which are solved in under a millisecond. The exception is $n = 17$, where the winning candidate required 2.19 seconds, and a total runtime of 47.4 seconds.

The solution for $n = 17$ is shown in figure 1.

6 Conclusions

In this report we presented a CSP model for the Rectangle Packing problem, implemented using the Pychoco constraint solver. The model relies on the *diff_n* global constraint to enforce non-overlap between squares, with domain tightening and symmetry breaking on the largest square to reduce the search space.

The minimum enclosing rectangle is found by enumerating candidate rectangles in order of increasing area and calling `pack` on each, guaranteeing optimality

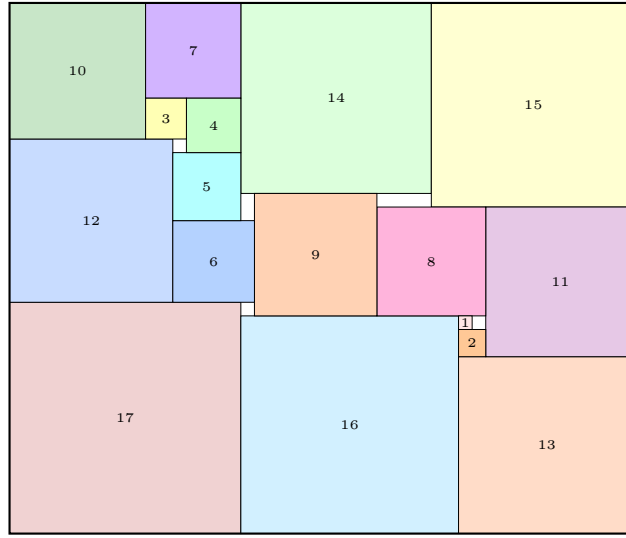


Fig. 1. Optimal packing for $n = 17$ in a 46×39 rectangle (area = 1794, fill = 99.5%). Each square is labelled with its size.

by construction. The total computational cost of this approach scales with both the number of candidates and the difficulty of each individual `pack` call, making it progressively more expensive for larger values of n .

Among the search strategies evaluated, *dom/Wdeg* consistently outperformed the alternatives, achieving the fewest failures and the lowest solving time.

References

1. Boussemart, F., Hemery, F., Lecoutre, C., Sais, L.: Boosting systematic search by weighting constraints. In: ECAI. vol. 16, pp. 96–97 (2004)
2. Simonis, H., O’Sullivan, B.: Search strategies for rectangle packing. In: International Conference on Principles and Practice of Constraint Programming. pp. 52–66. Springer (2008)